

Logistic Regression

It's not actually regression.

The name is misleading, but the concept is incredibly powerful. Find out why this basic algorithm is the backbone of classification systems globally.

- 🔴 Classification
- 📊 Sigmoid Function
- 🚦 Decision Thresholds
- 📧 Spam Filter Analogy

Why this matters

It's one of the most common questions in machine learning interviews. Proving you understand the math and the intuition behind its boundary logic makes a huge difference. Let's dive in.

Classification vs Regression

Linear Regression predicts numerical steps. **Logistic Regression** is used when you need to assign data points to discrete categories.



Linear Regression

TARGET GOAL

Predict continuous numerical values

OUTPUT BOUNDS

$-\infty$ to $+\infty$ (Unbounded)

EXAMPLES

Salary, House Prices, Temperature



Logistic Regression

TARGET GOAL

Predict probability of discrete classes

OUTPUT BOUNDS

Bounded between 0 and 1 ($0.0 \leq p \leq 1.0$)

EXAMPLES

Spam vs Not Spam, Pass vs Fail

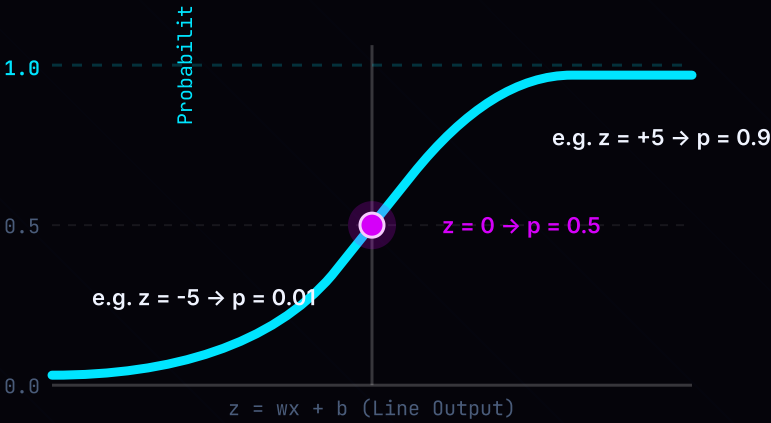
Why the name regression then? Mathematically, it works similarly to Linear Regression. It models a continuous value internally ($z = wx + b$), but then applies a mathematical function to **squash** it into a probability.

Summary: Logistic regression is a classification model that calculates probability boundaries instead of linear slopes.



The Sigmoid Function

To force the unbounded line output z to become a probability, we pass it through the **Sigmoid (Logistic) Function**:



$\sigma(z) = \frac{1}{1 + e^{-z}}$ Center Threshold (0.5)

As $z \rightarrow +\infty$

The value of e^{-z} drops to 0. The denominator becomes 1, squashing the output probability closer to **1.0**.

As $z \rightarrow -\infty$

The value of e^{-z} grows extremely large. The denominator approaches infinity, squashing the probability to **0.0**.

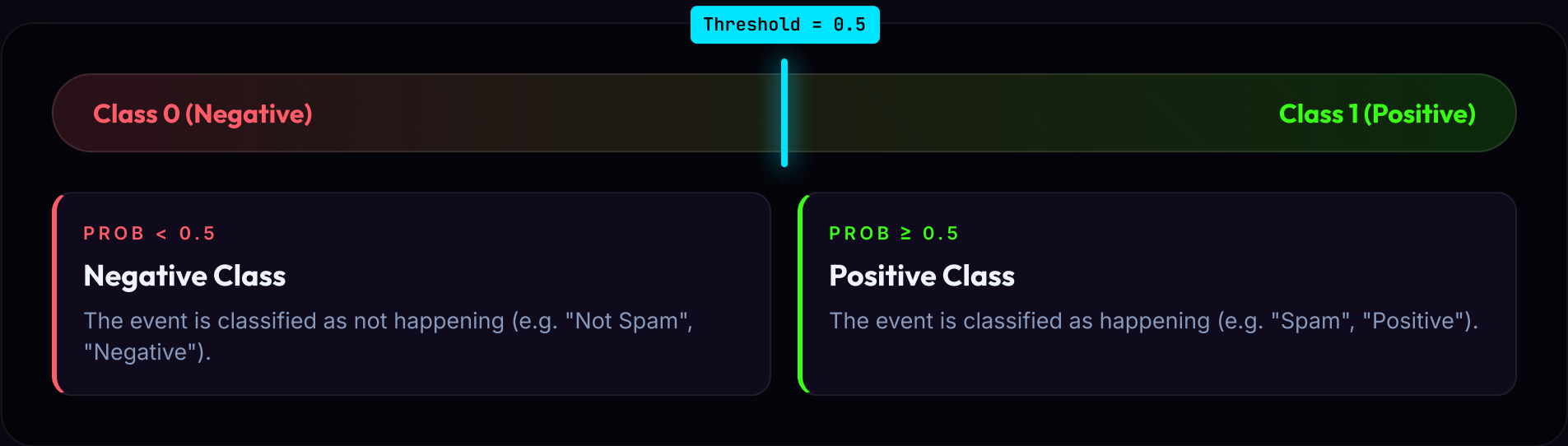
Squashing

No matter how positive or negative the raw score is, the Sigmoid function maps it into a neat probability between 0% and 100%.



The Decision Threshold

Sigmoid output is just a probability (e.g. `0.75` chance of spam). To make a final binary choice, we apply a **Decision Threshold** (usually `0.5`):



Tuning the Boundary

The threshold doesn't have to be 0.5. In medical diagnostics, you might lower the threshold (e.g. 0.2) to catch as many positive cases as possible, accepting more false positives. In spam filters, you might raise it (e.g. 0.8) to ensure important emails are never blocked by accident.

Tuning

Changing the threshold from 0.5 shifts the balance between precision and recall depending on business needs.



The Spam Filter Pipeline

Here is exactly how Logistic Regression classifies an incoming email as Spam or Not Spam step-by-step:



1. Incoming Email Message

An email arrives containing text like: "Get free coins immediately!"



2. Model Feature Extraction

Model counts triggers (x_1 =has 'free', x_2 =capital letters). Weighs them (w_1 , w_2) to get raw score (z).

$z = wx+b = 2.65$



3. Sigmoid Squashing

The raw score ($z=2.65$) passes through the Sigmoid function to generate a probability curve.

$p = \sigma(2.65) = 0.93$



4. Threshold Check & Classify

Comparing $0.93 \geq 0.5$. Since it's higher than the threshold, it is routed to the Spam folder.

Class 1: SPAM



Vinod Bavage

Machine Learning · AI

</> vinodbavage31

Day 12 / 60

Implementation

Here is how you train a Logistic Regression model and look at both class predictions and raw probabilities in Python:

```
# 1. Import and initialize the classifier
from sklearn.linear_model import LogisticRegression
model = LogisticRegression()

# 2. Fit model on training data
model.fit(X_train, y_train)

# 3. Predict the probability of Spam (Class 1)
probabilities = model.predict_proba(X_test)[: , 1]
# Outputs array of floats between 0.0 and 1.0 (e.g. [0.93, 0.12])

# 4. Predict binary class directly (using 0.5 threshold)
predictions = model.predict(X_test)
# Outputs array of binary integers (e.g. [1, 0])
```

Method Breakdown

- `predict()`: Applies the default 0.5 threshold automatically and outputs integer predictions (0 or 1).
- `predict_proba()`: Gives you the raw probability estimates for each class, which you can use to apply a custom decision threshold (e.g. `probs > 0.8`).

Implementation Tip: When evaluating model thresholds, always save the raw outputs of `predict_proba()` so you can evaluate the threshold curve.



Follow Vinod Bavage for Day 13 →



Tomorrow: Evaluations: Precision, Recall, and the F1-Score

Day 12 Recap:

🎯 Bounded output between 0 and 1

🚦 Default decision boundary = 0.5

📧 Used for binary spam detection

📐 Squashed via Sigmoid activation

⚖️ Binds linear combinations to curves

🚀 Simple, fast, and easy baseline

#MachineLearning

#DataScience

#LogisticRegression

#Classification

#VinodBavage

VINOD BAVAGE



Vinod Bavage
Machine Learning · AI

</> vinodbavage31

Day 12 / 60